

EE-2003

Computer Organization and Assembly Language

Week 09 – Lecture 17

Department of Computer Science/Software Engineering/ Artificial Intelligence
FAST-NUCES Peshawar

Outline

- 1 Quick Review
- 2 Bitwise Logical Operations
- 3 Masking Operations
- 4 Example 4.3 – 16-bit Multiplication Using TEST

Quick Review: What was in the Last Lecture?

- Shift and Rotate Instructions
- Extended Shifting & Operations
- 4-bit, 8-bit & 16-Bit Multiplications

Bitwise Logical Operations

- The 8088 processor provides a few logical operations that operate at the **bit level**.
- The four basic logical operations are:
 - ① AND
 - ② OR
 - ③ XOR (Exclusive OR)
 - ④ NOT
- These operations are **bitwise**, meaning they act on corresponding bits of operands in parallel.

Understanding Bitwise Operations

- Example: AND AX, BX
- Operation is applied on corresponding bits of AX and BX.
- Each bit position performs its own logical operation independently.
- Example:
 - Bit 0 of AX = Bit 0(AX) AND Bit 0(BX)
 - Bit 1 of AX = Bit 1(AX) AND Bit 1(BX)
 - ... and so on for all 16 bits.

AND Operation

- Performs logical **bitwise AND** between two operands (byte or word).
- Result bit is set only if **both** corresponding bits are 1.
- Example instructions:
 - AND AX, BX
 - AND BYTE [mem], 5

Bit A	Bit B	Result (A AND B)
0	0	0
0	1	0
1	0	0
1	1	1

OR Operation

- Performs logical **bitwise inclusive OR**.
- Result bit is set if **either or both** corresponding bits are 1.
- Example instructions:
 - OR AX, BX
 - OR BYTE [mem], 5

Bit A	Bit B	Result (A OR B)
0	0	0
0	1	1
1	0	1
1	1	1

XOR Operation (Exclusive OR)

- Performs logical **bitwise exclusive OR**.
- Result bit is set if bits of operands are **different**.
- Example instructions:
 - XOR AX, BX
 - XOR BYTE [mem], 5
- **Important property:** XOR is **reversible** — used in cryptography, image processing, and graphics.

Bit A	Bit B	Result (A XOR B)
0	0	0
0	1	1
1	0	1
1	1	0

NOT Operation

- **NOT** inverts the bits (forms the one's complement) of the operand.
- It is a **single-operand** instruction.
- Example instructions:
 - NOT AX
 - NOT BYTE [mem]
- Each bit is flipped: $0 \rightarrow 1$, $1 \rightarrow 0$.

Original Bit	Result (NOT Bit)
0	1
1	0

Summary

- All four logical operations (AND, OR, XOR, NOT) are bitwise and operate in parallel on all bits.
- They are essential for:
 - Masking and clearing bits
 - Setting or toggling specific bits
 - Encryption and checksum algorithms
 - Digital image and signal processing

4.6 Masking Operations

Selective Bit Clearing (Using AND)

- Used to make selective bits zero in the destination operand.
- Source operand holds a mask:
 - 1 → retain old value
 - 0 → clear bit
- Example: To clear the lower nibble, use mask = F0. The upper nibble retains its value.

Selective Bit Setting (Using OR)

- Used to set selective bits to 1.
- Mask bits:
 - 1 → set bit
 - 0 → retain original value
- Example: To set lower nibble, mask = 0F.

Masking Operations (continued)

Selective Bit Inversion (Using XOR)

- XOR can invert selective bits.
- Mask bits:
 - 1 $\rightarrow$ invert bit
 - 0 $\rightarrow$ retain bit
- Example: To invert lower nibble, mask = 0F.
- Compare with NOT, which inverts all bits. XOR allows **partial inversion**.

Selective Bit Testing (Using AND / TEST)

- AND can test bits but destroys the mask.
- TEST = non-destructive AND:
 - Sets flags only (does not alter operands).
 - ZF = 1 $\rightarrow$ bit clear, ZF = 0 $\rightarrow$ bit set.
- Enables testing any bit directly, not just sequentially.

Example 4.3 – 16-bit Multiplication Using TEST (Part 1)

```
[org 0x0100]
    jmp  start

multiplicand: dd    1300          ; 16-bit multiplicand (32-bit space)
multiplier:   dw    500           ; 16-bit multiplier
result:       dd    0             ; 32-bit result

start:        mov    cl, 16        ; initialize bit count
              mov    bx, 1         ; initialize bit mask

checkbit:     test   bx, [multiplier] ; test lowest bit
              jz     skip          ; skip addition if bit = 0
              mov    ax, [multiplicand]
              add    [result], ax   ; add lower word
              mov    ax, [multiplicand+2]
              adc    [result+2], ax ; add upper word
```

Example 4.3 – 16-bit Multiplication Using TEST (Part 2)

```
skip:      shl  word [multiplicand], 1
           rcl  word [multiplicand+2], 1 ; shift multiplicand left
           shl  bx, 1                    ; shift mask to next bit
           dec  cl                        ; decrement bit count
           jnz  checkbit                  ; repeat if bits left

           mov  ax, 0x4c00                ; terminate program
           int  0x21
```

Explanation / Observations:

- TEST checks a bit without modifying BX or memory.
- BX acts as a shifting mask—moves left each iteration.
- Counting can be skipped; loop may end when BX = 0.
- In debugger: BX and memory stay unchanged after TEST.
- Final result = 0009EB10 – same as earlier example.
- Demonstrates bitwise masking efficiency in assembly.

Thank You!

Any Questions?

- **Topics Covered:** Bitwise Logical Operations (AND, OR, XOR, NOT) and Masking Techniques.
- **Key Lessons:**
 - **Bitwise Operations:** Operate in parallel on all bits — foundation for low-level logic design.
 - **Masking:** Enables selective bit **clearing, setting, inversion, and testing**.
 - **TEST Instruction:** Performs non-destructive bit testing without altering operands.
 - **Practical Relevance:** Used in control registers, encryption, error detection, and data manipulation.

“Bitwise logic builds the bridge between hardware precision and software intelligence.”